Class 3: Machine Learning Foundations

From Zero to AI Agents · Fall 2026



Scan to open the form

A quick recap

LAST CLASS

Building software with AI

We used AI to turn an idea into working software.

THIS CLASS

Understanding how AI works

We'll learn how models learn from data and make predictions.



Why This Class?

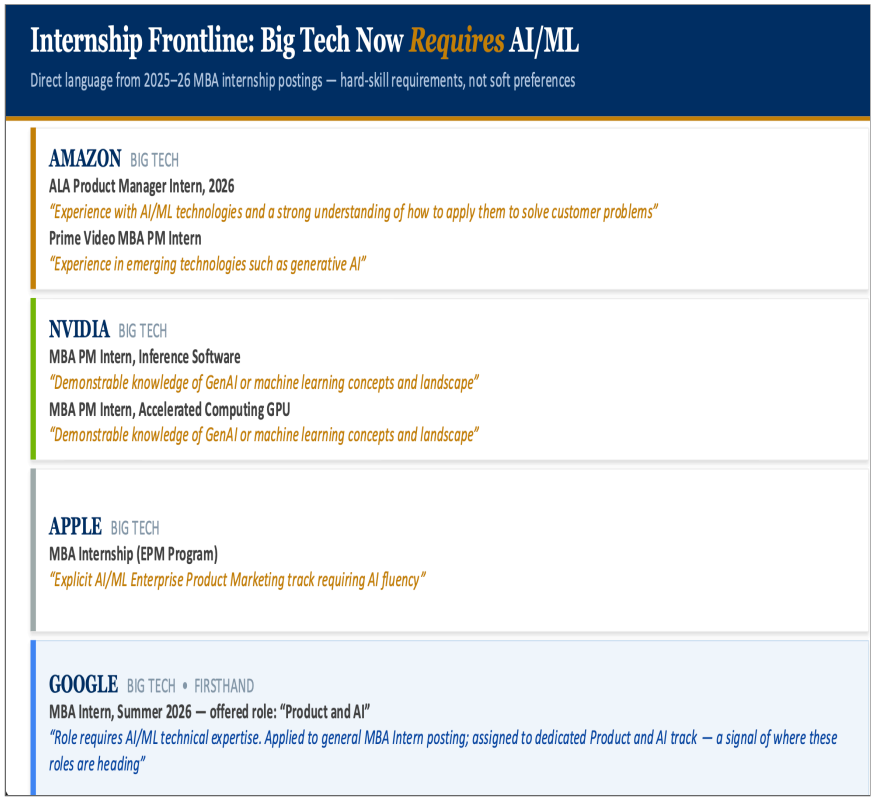
Industry Expectations for MBAs Have Changed

Business roles increasingly connect **product, strategy, data, and technical execution**.

- Evaluating an AI proposal means understanding what the model learns, what data it needs, and how its performance is measured.
- Building an in-house model involves choices about labels, costs, errors, and deployment. Those are management decisions too.
- When someone says “the model is overfitting,” you should be able to ask the next useful question.

The goal is technical fluency: enough understanding to make decisions about AI, including when to use something simpler.

Big Tech Now *Requires* AI/ML



Examples collected for the course from 2025–26 internship postings.

Not Just Tech: Finance, Consulting, Healthcare Too

Internship Frontline: Finance, Consulting & Healthcare

AI requirements extend well beyond Big Tech — finance, consulting, and healthcare now list them explicitly

CAPITAL ONE FINANCE

What We Need To See:

"platforms such as data and machine learning"

ACCENTURE STRATEGY CONSULTING

Preferred Qualifications:

"Bonus points for... interest in emerging technologies (e.g., Generative AI)"

AMGEN HEALTHCARE

Preferred Qualifications:

"Exposure to AI concepts such as machine learning, natural language processing, or decision intelligence"

Source: Verbatim from Capital One, Accenture, Amgen internship postings (2025–26 cycle)

The same vocabulary appears beyond technology companies.

Why AI Instead of Code?

Last Class We Wrote the Rules

Software

```
if invoice_is_overdue:
    block_discount()
```

We specify the rule. The app applies it to each customer.

Machine learning

```
past usage + renewal outcomes
            ↓ train
       learned model
            ↓ predict
renewal estimate for a new account
```

We choose the inputs and objective. Training fits the model's settings.

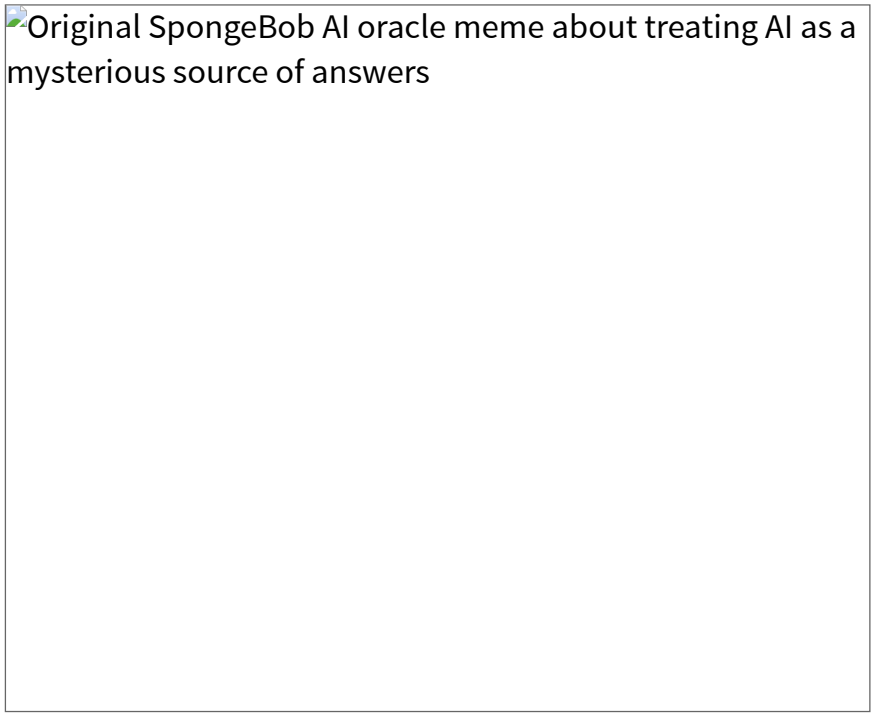
Why Can't We Just Code All the Rules?

Chess has simple rules. A finite board. A finite set of pieces. Couldn't we just program every possible move?

	Number	Scale
Possible chess games	10^{120}	a 1 followed by **120** zeros
Atoms in the universe	10^{80}	a 1 followed by 80 zeros

Chess has **a trillion trillion trillion times** more possible games than there are atoms in the universe. You cannot brute-force chess. This is why we need AI.

AI Feels Like an Alien Oracle…



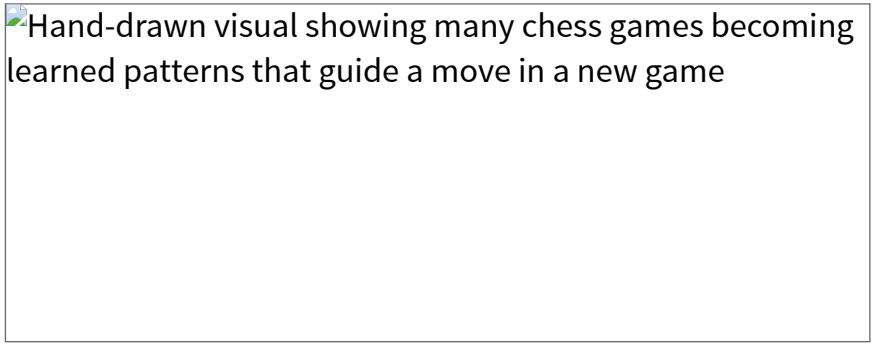
Underneath: **statistics, linear algebra, and optimization**, running at a scale we could not work through by hand.

Condensation of Knowledge

A chess grandmaster does not memorize every possible game. They develop **intuition from patterns across many games.**

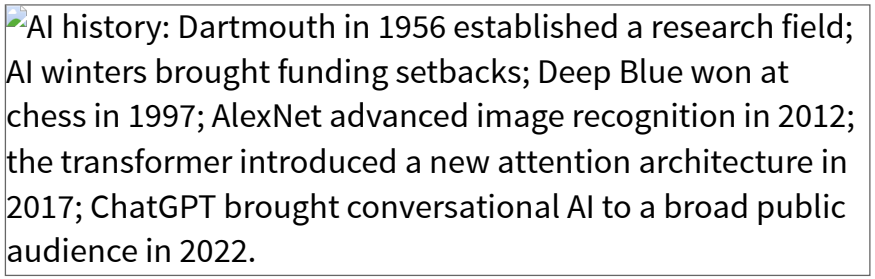
- A learned model captures useful patterns from its training examples in its **parameters**.
- Those parameters let it make predictions about **new examples**.
- That only helps if the learned patterns still apply when the situation changes.
- **Predicting an outcome does not tell us how to change it:** knowing who might leave does not tell us how to make them stay.

  This is the intuition behind learning: turn experience into a reusable mathematical model.



AI Is Older Than You Think

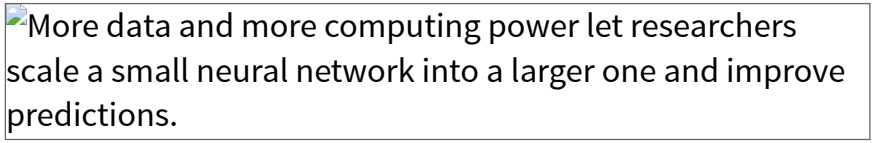
AI: A 70-Year Journey



AI grew through rules, search, and learning, long before chatbots.

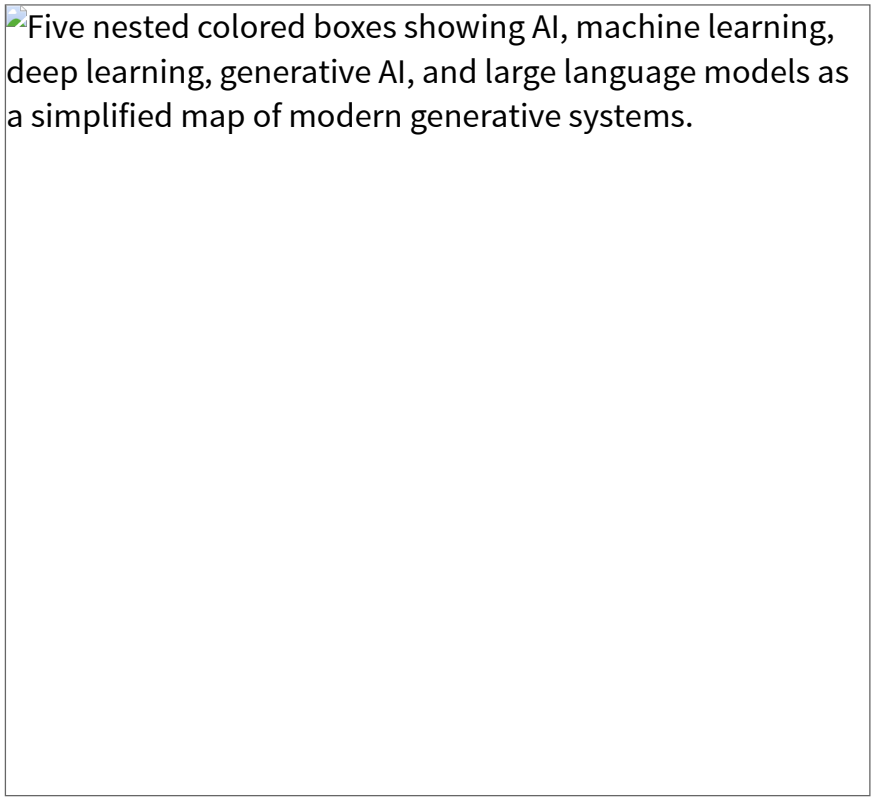
Scale Became a Driver of AI Progress

- **Earlier AI:** experts often designed rules and features for each task.
- **Modern AI:** designs like transformers can improve with **more data, more compute, and larger models**, without a new algorithm each time.



Scaling laws help predict how much improvement that extra scale can buy.

The AI Landscape



Today: **machine learning**. Next class: **deep learning** and transformers.

Today's Route

First half	Second half
AI history, the landscape, and why learning helps	Check whether learning generalizes
Define the prediction and prepare data	Compare models and evaluate errors
Fit a model by reducing loss	Find clusters without labels
Train a digit classifier in Colab	Choose actions through reinforcement learning
	Train a Grid World policy, then start Pac-Man

The question throughout: what did the model learn, and how would we know?

You Already Know the Math Behind AI

Machine learning builds on **statistics, probability, linear algebra, and optimization**.
Your background in **econ, stats, or finance** gives you a foundation for understanding how these models work.
Remember **Data and Decisions**? You have already worked with the core ideas:

You already know	In machine learning, that's
Probability and distributions	Describing uncertainty and noise
Linear regression	A machine learning algorithm you already know
Optimization / minimizing error	How training finds better model parameters
Sampling and validation	Checking whether results hold for new data

Machine learning is applied math at scale. Your MBA has already introduced the fundamentals.

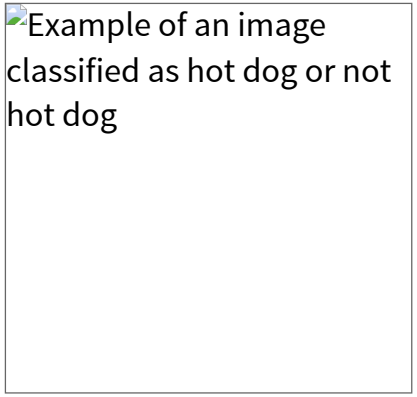
Three Ways a Machine Can Learn

Learning	What we give it	What it learns	Today's example
Supervised	Inputs + known answers	Predict an answer	Read a handwritten digit
Unsupervised	Inputs without answers	Find structure	Group similar customers
Reinforcement	An environment + rewards	Choose actions over time	Navigate a grid, then Pac-Man

Neural networks are a type of model. They can be used in all three forms of learning. We open them up in Class 4.

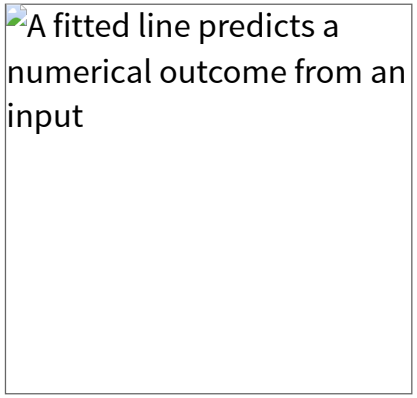
Classification, Regression, and Ranking

Classification: which category?



Spam or not spam. Digit 0 through 9. Churn or stay.

Regression: what number?



Sales next week. Delivery time. House price.

Ranking puts candidates in order, such as which leads sales should contact first.

Labels Are the Answers We Learn From

Supervised learning starts with examples paired with known answers.

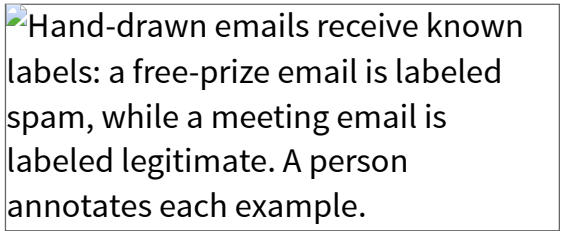
Features (x)

What the model can see: words, links, and sender information.

Label (y)

The answer attached to that example: **spam** or **legitimate**.

Labeling can come from **human review** or **recorded outcomes**.



Classification: learn a category. **Regression:** learn a number, such as a house's recorded sale price.

For a new example, the answer is unknown. The model makes a **prediction**.

Training a Model

A Model Has Adjustable Parameters

$$\text{predicted revenue} = w \times \text{ad spend} + b$$

The input: ad spend.
The parameters: slope w and intercept b.
Training chooses parameter values that fit historical examples.

```
def predict(ad_spend, w, b):
    return w * ad_spend + b
```

Using the fitted model on new inputs is **inference**.

You already know this as **fitting a regression**. ML uses the same idea for many kinds of prediction.

Loss Measures Prediction Error

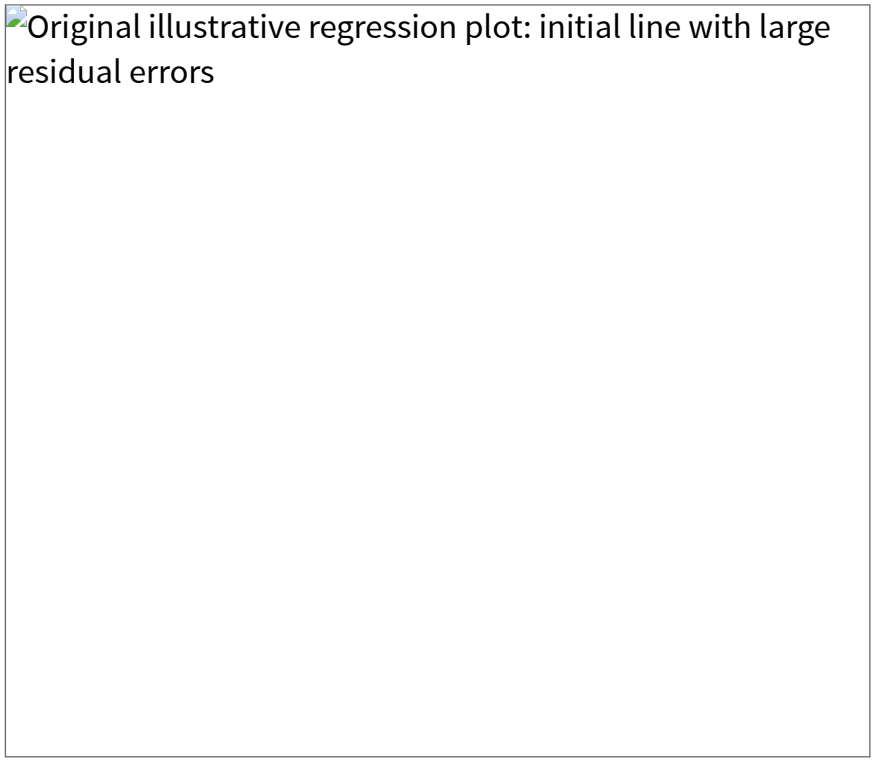
Toy revenue data, in **thousands of dollars**:

Actual revenue	Prediction	Error	Squared error
10	8	−2	4
20	23	+3	9
30	25	−5	25

$$\text{mean squared error} = \frac{4 + 9 + 25}{3} \approx 12.7$$

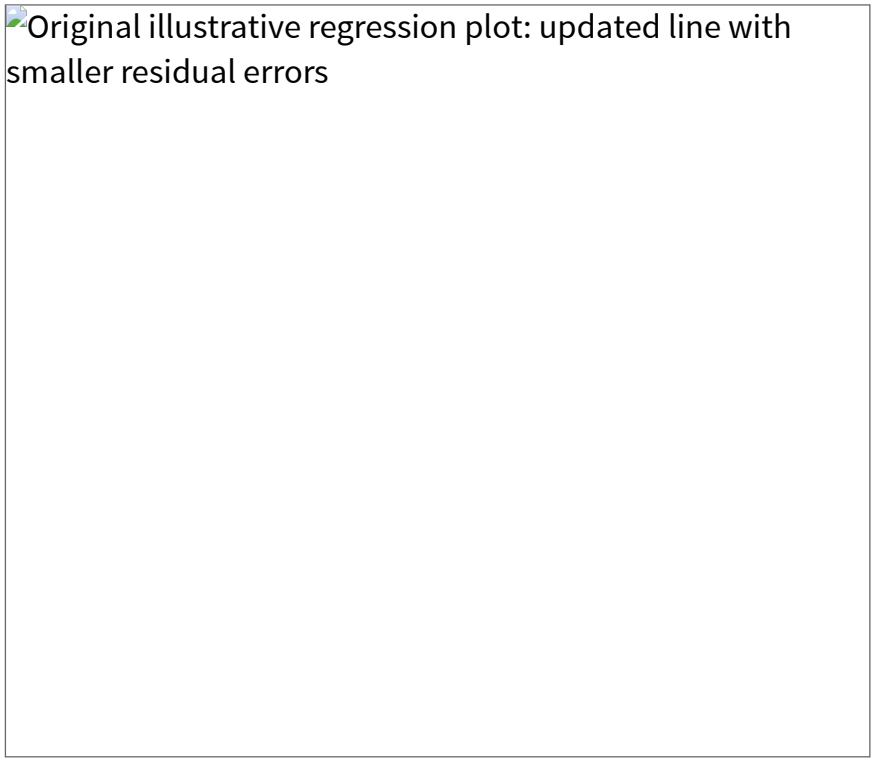
Training tries to reduce this average. Squaring makes large misses count more.

Training Starts With a Poor Guess



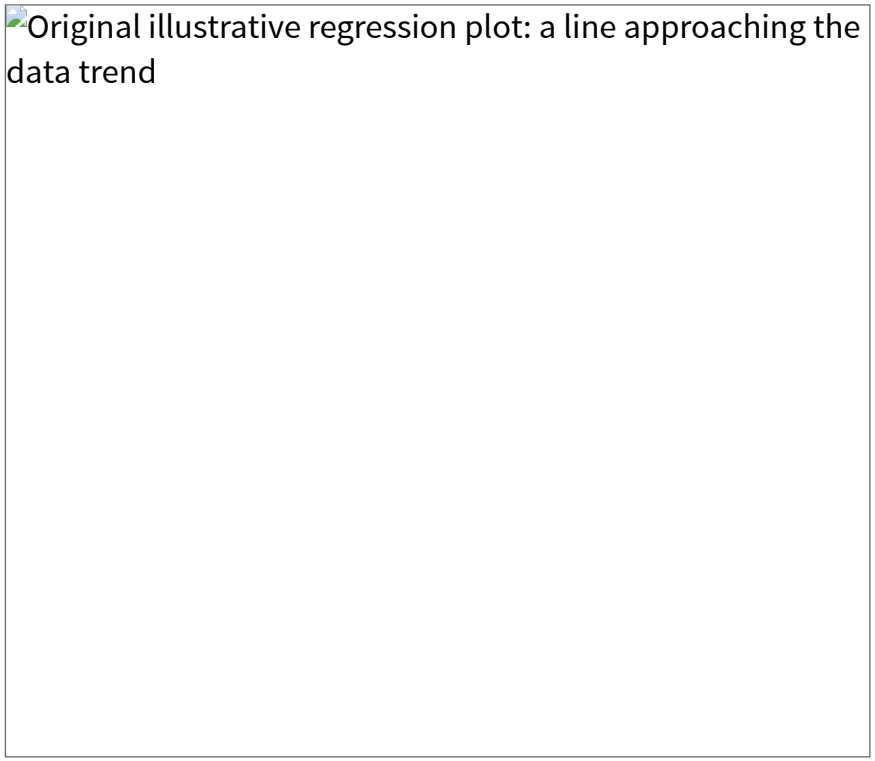
Start: the predictions miss most of the observations. The dashed lines show the gap between predictions and observations. Loss values in this sequence are illustrative.

Adjusting the Line



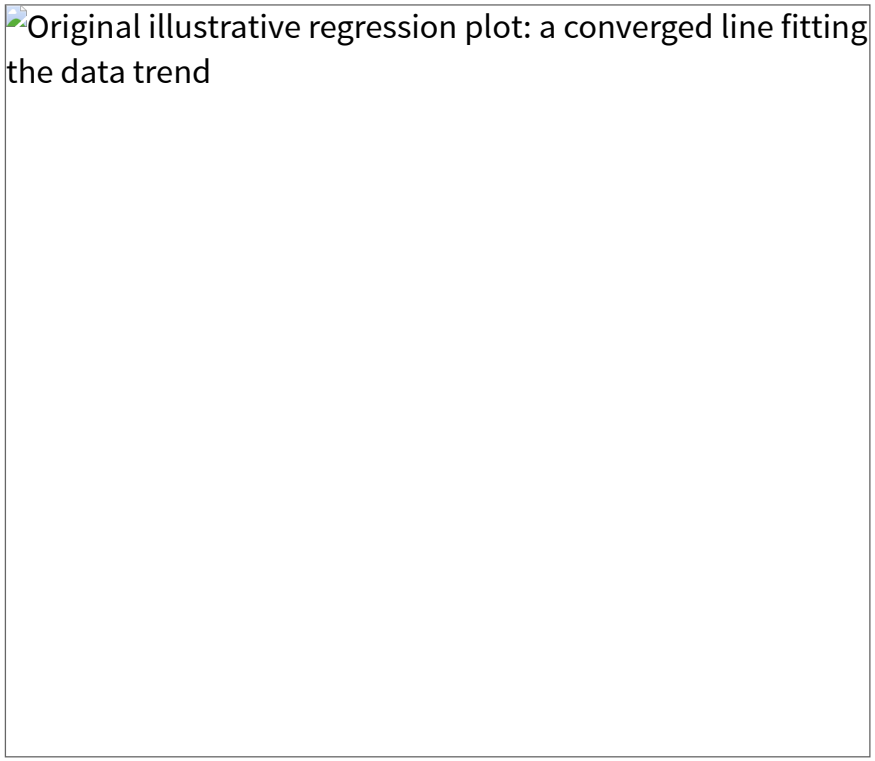
Update: change the slope and intercept to reduce loss. Loss drops from **847 → 312**. The fit improves, but the predictions still miss the observations.

Repeated Updates Improve the Fit



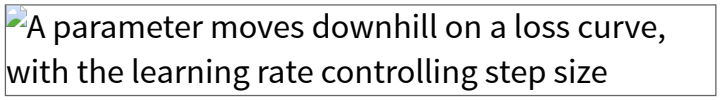
Keep adjusting: the errors get smaller.
Loss drops from **312 → 58**. The line is getting closer to the data trend.

The Fit Converges



Converge: further updates make little difference.
Loss reaches **12**. **That is training:** measure error, adjust parameters, repeat. Next, we will make the updates ourselves.

Gradient Descent Updates the Parameters



Predict → measure loss → compute the gradient → take a small downhill step → repeat.

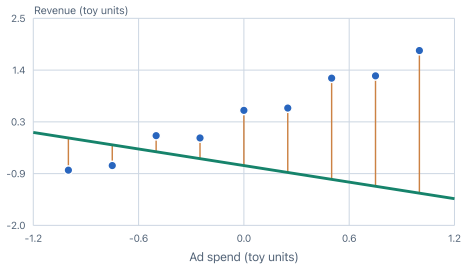
The **gradient** tells us how loss changes with each parameter.

The **learning rate** controls the step size.

Try It: Fit the Line

Change the learning rate, then step through training. Watch the **line, residuals, and loss** change together.

Learning rate | Moderate · 0.3 | One update | 25 updates | Reset



Blue dots: observations · green line: prediction · orange lines: errors

The fitted model

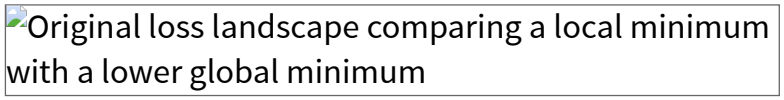
Updates	0
Slope w	-0.60
Intercept b	-0.70
Mean squared error	2.736

Start with a poor guess. Each update uses all nine observations.

Synthetic data in toy units. Learning-rate choices depend on the scale of the inputs.

Try: reset, select **Too large**, and take a few steps. Explain why more training does not help that run.

Local Minima



The line we just fitted has a convex loss: one bowl, with no competing local valleys.
More complicated models can have several valleys. Gradient descent may settle in a **local minimum**; different starting points can lead to different fits.

Classification Learns Probabilities

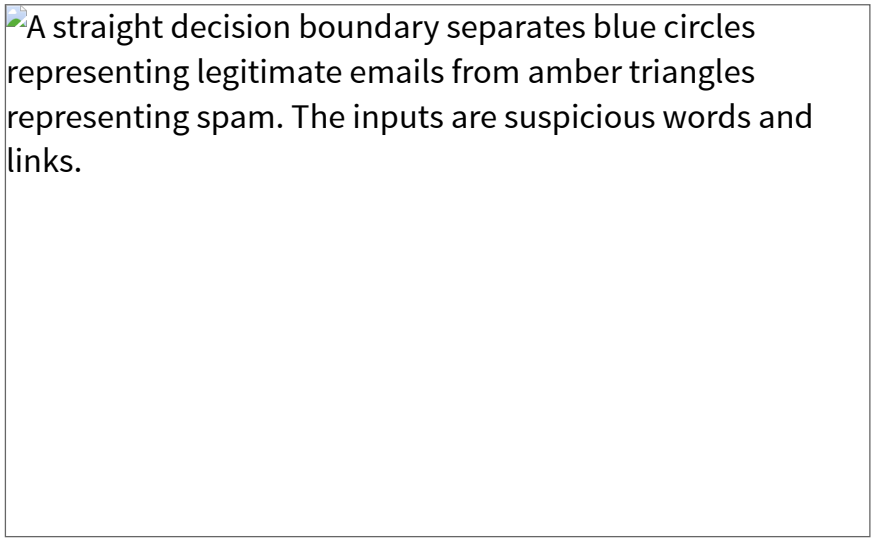
For an account that **did renew**, compare three predictions:

Predicted renewal probability	Cross-entropy loss, $-\log(p)$
90%	0.11
50%	0.69
1%	4.61

A confident wrong answer gets a much larger penalty. For an account that did not renew, the loss uses $-\log(1-p)$.
Logistic regression turns a weighted sum of features into a probability with a sigmoid. Despite its name, it is used for classification.

Classification Learns a Decision Boundary

Each point is a labeled email. The boundary separates **where the model predicts each class**.



Binary: spam or legitimate. **Multiclass:** one of ten handwritten digits, 0 through 9.
Logistic regression estimates a class probability. A **threshold** turns it into a label:
70% estimated spam probability + a 50% threshold → **predict spam**.

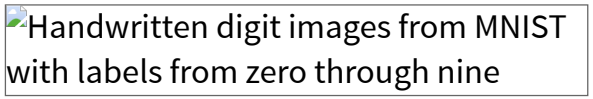
Training and Inference Use Different Information

	During training	During inference
Inputs	Historical features	Features for a new case
Known answers	Labels used to compute loss	Usually unavailable yet
Parameters	Updated to reduce loss	Held fixed for prediction
Result	A fitted model	A score, category, or number

An app making a prediction is **not automatically learning from that interaction**. Updating the model requires a training process.

Mid-Class Project: Read Handwritten Digits

MNIST Turns Pictures Into Training Examples



28 × 28 pixels = 784 input features. Each image has one label: its digit.

The notebook fits ten weighted scores: **784 × 10 weights + 10 biases = 7,850 parameters**.

Run the Classifier in Colab

Open the MNIST notebook →
Choose **Runtime → Run all**. It runs in your browser; nothing to install.

1. Find the **weights**, **loss**, and **update step** in the code.
2. Record the test accuracy from your run.
3. Inspect three mistakes. What makes those images difficult?
4. Run the shifted-image experiment. Record what changed.

Scan to open the MNIST notebook in Google Colab

Scan to open the notebook

Done: show one correct prediction, one mistake, and one limitation of the model. This is the in-class learning exercise.

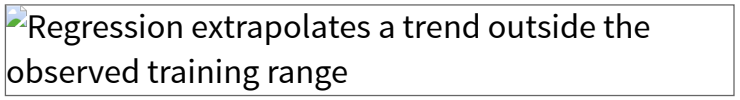
A Small Image Shift Changes the Task

The notebook moves test digits sideways while keeping their labels the same.

Test input	What changes?	What to record
Original digit	Same placement as training	Your starting accuracy
Shifted 2 pixels	Ink lands on different inputs	New accuracy
Shifted 5 pixels	A larger change in position	New accuracy and an example

The model learned **which pixels predict each digit in the training distribution**. It was not designed to recognize a shape equally well wherever it appears.
Keep this failure in mind. Class 4 introduces networks that learn useful image features.

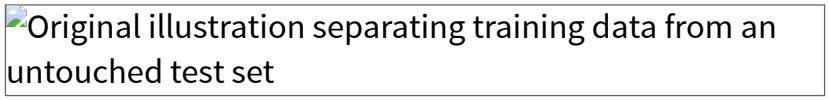
Extrapolation Uses the Model Beyond Its Data



The shifted digits showed a change in input conditions.
Extrapolation is a related risk: predicting beyond the observed training range.
A trend for small purchases may not hold for very large ones.
Flag inputs outside that range and check performance under new conditions.

Generalization and Model Choice

Keep a Test Set Outside Development



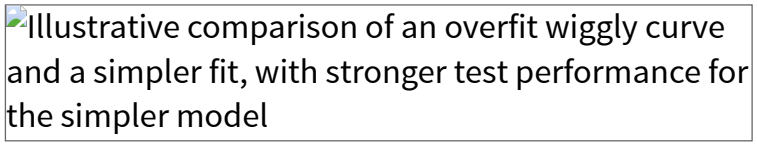
If we keep changing the model after seeing its test score, that score starts guiding development. Keep the final test set untouched until our choices are fixed.
Use **validation data** to choose settings. The illustrated 80/20 split is only an example; the split should reflect future deployment.

Separate Fitting, Choosing, and Testing

Training data	Validation data	Test data
Fit weights or tree splits	Choose model, settings, threshold	Estimate final performance
Can be revisited during fitting	Can guide development	Keep untouched until choices are fixed

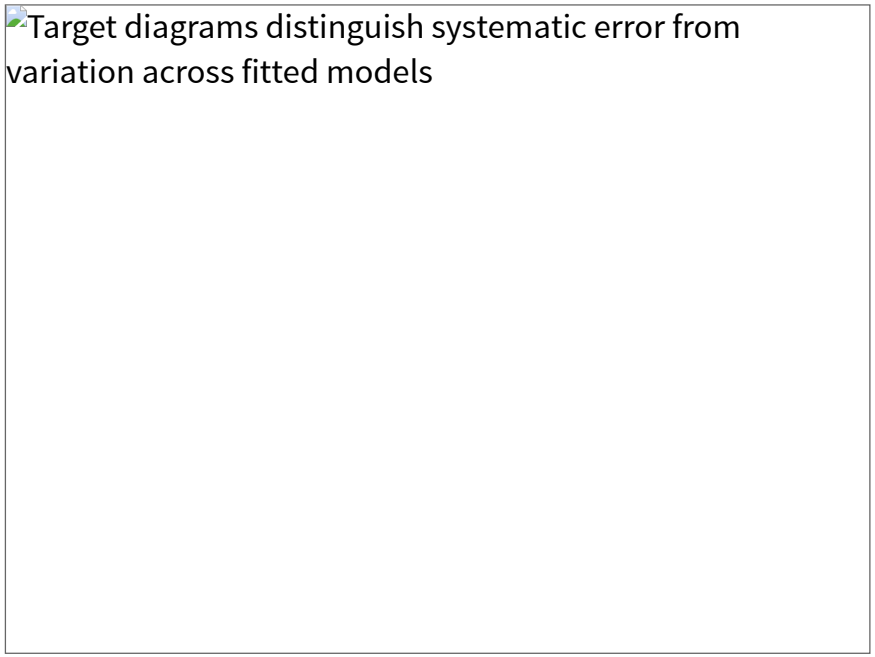
For future predictions: train on older cases, validate on later ones, test on the newest held-out period.
Keep related records together when necessary. The same person’s near-duplicate images in both sets can make the score misleading.

Overfitting Fits Noise Along With Signal



Underfitting: the model is too limited to capture the pattern.
Overfitting: it fits training details that do not carry over.
Compare **training and validation performance**, not just training loss. Numbers in the diagram are illustrative.

Bias and Variance



Bias: systematic error from simplifying assumptions. **Variance:** how much the fitted model changes with the training sample.
Too rigid: miss the pattern. **Too flexible:** chase the sample.
Choose complexity using validation; noise also limits prediction quality.

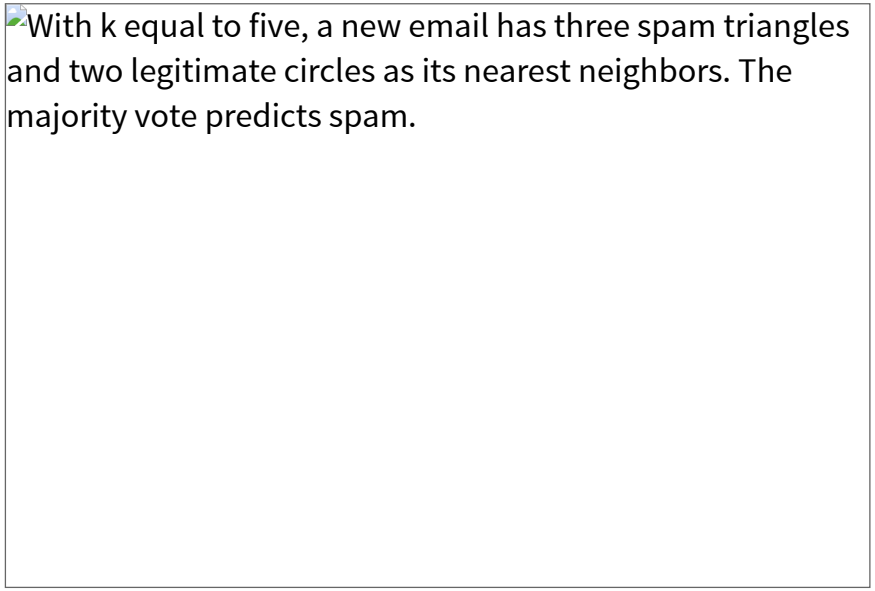
Control Complexity, Then Validate

What you observe	What to investigate
Poor training and validation results	Weak features, too little capacity, incomplete training
Good training, poor validation	Overfitting, leakage, or a mismatched split
Good validation, poor live results	Distribution shift or a production data problem

Regularization discourages overly complex fits. Examples: penalize large weights, limit tree depth, or stop training when validation stops improving.
Parameters are fitted during training. **Hyperparameters**, such as learning rate and tree depth, are settings we choose using validation.

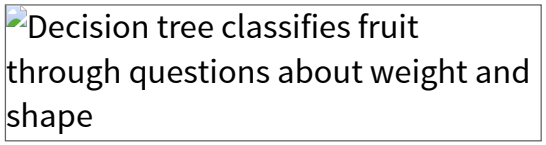
K-Nearest Neighbors Takes a Vote

Find the k **most similar labeled examples**. For classification, let them vote.



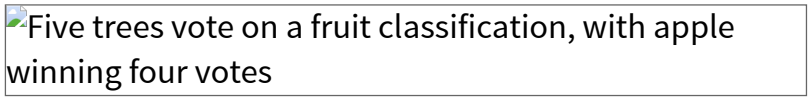
$k = 5$**:** three neighbors are spam, two are legitimate → **predict spam**.
“Near” depends on the features and their scale. For **regression**, average the neighbors’ numbers instead.

Decision Trees Learn a Flowchart



Each **split** asks a question about a feature.
Training chooses questions that separate the observed outcomes.
A **leaf** gives the prediction. For regression, it can give a mean value.
Deeper trees fit more detail and can overfit.

Random Forests Combine Different Trees



Train trees on resampled data with random feature choices.
Combine their predictions by voting or averaging.
Different trees make different errors. Combining them can reduce variance, at the cost of a more complicated explanation.

Features Translate Raw Records Into Signals

Raw record	Possible feature	Why it might help
Login timestamps	Days since last activity	Separates active from lapsed use
Purchases	Amount ÷ customer's usual amount	Measures an unusual purchase
Income and rent	Rent-to-income ratio	Makes the relationship explicit

Compute features using only information available at prediction time.

For images, what would the equivalent columns be? Edge direction? Loops? Curves? **Learning those representations is the next class.**

Compare Against a Simple Baseline

Problem	First comparison	What else to try
Predict a number	Historical mean or linear regression	Trees and ensembles
Predict a category	Majority class or logistic regression	Trees and ensembles
Group unlabeled rows	Inspect a scatterplot	Clustering
Choose actions over time	Random or hand-written policy	Reinforcement learning

A more complex model earns its place through **better held-out results at an acceptable cost**.

Evaluating Predictions

True Positives and False Alarms

First define **positive = spam**. A **confusion matrix** compares predictions with known labels.

Known label ↓ Prediction →	Predicted spam (+)	Predicted legitimate (−)
Actually spam (+)	**True positive (TP)** Spam caught.	**False negative (FN)** Spam reaches your inbox.
Actually legitimate (−)	**False positive (FP)** Your job offer goes to spam.	**True negative (TN)** Your job offer reaches your inbox.

Positive / negative: what the model predicted. **True / false:** whether it was right.

Which costs more: a missed spam email or a lost job offer?

Accuracy Can Hide the Error You Care About

Toy dataset: **100 transactions, including 5 frauds**.

Model	Correct predictions	Accuracy	Frauds caught
Always predict legitimate	95	**95%**	**0 of 5**
Flag 10, including 4 frauds	93	**93%**	**4 of 5**

The second model catches fraud, but also flags **6 legitimate payments**.
Which is better depends on the cost of missed fraud and false alarms.

Precision and Recall Count Different Mistakes

The same 100 transactions:

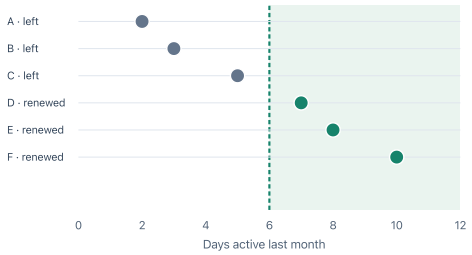
	Flagged as fraud	Not flagged
Actually fraud	**4** true positives	**1** false negative
Actually legitimate	**6** false positives	**89** true negatives

Precision = 4 / 10 = 40%
Of the flagged payments, how many really were fraud?
Recall = 4 / 5 = 80%
Of the actual frauds, how many did we catch?

Try It: Choose a Renewal Threshold

Predict **renewal** when usage is at or above the threshold. Here, **positive means renewed**.

Threshold **6 days** [Add messy account] [Reset]



Green region: predict renewal · orange outline: wrong prediction

Positive = renewed

Actual	Predict renew	Predict leave
Renewed	3 TP	0 FN
Left	0 FP	3 TN

Accuracy	**100%**
Precision	**100%**
Recall	**100%**

Try 4, 6, and 8 days. Which accounts change prediction?

Try 4, 6, and 8 days. Then add the messy account. Explain which mistake your chosen threshold accepts. This exercise is ungraded.

A Prediction Still Needs a Decision Policy

Model output	Decision we still have to make
High estimated churn risk	Call, discount, investigate, or do nothing?
High fraud score	Block automatically or route to review?
An uncertain case	What context does a reviewer need?

Predicting who leaves does not tell us whether a discount will make them stay. Test the intervention separately. Check error rates across relevant groups, give reviewers authority to override, and monitor outcomes after launch.

Unsupervised Learning

Clustering Finds Groups Without Outcome Labels

k-means groups similar points around k centers, called **centroids**.

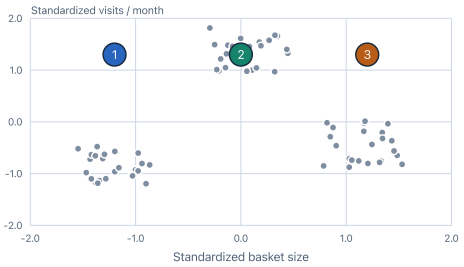
1. Start with k centers.
2. Assign each point to its nearest center.
3. Move each center to the mean of its assigned points. Repeat.

You choose the features, their scales, and k. The algorithm finds a grouping; **you decide whether it is useful**.

Try It: Watch the Clusters Form

Each point is a synthetic customer. The axes are standardized so both features contribute to distance.

Groups k = 3 Assign points Run to convergence Reset



Assign, then move

Customers	72
Completed iterations	0
Within-cluster squared distance	—

Numbered markers are starting centers. First assign the unlabeled gray points.

Both features are standardized. Lower distance with a larger k does not prove the extra groups are useful.

Try: compare $k = 2$ with $k = 3$. Does choosing more groups prove more customer segments exist?

Compression and Anomaly Detection

Compression

PCA combines correlated columns into fewer dimensions that preserve as much variance as possible. Useful for plotting and reducing inputs. Preserving variance does not guarantee preserving the signal your task needs.

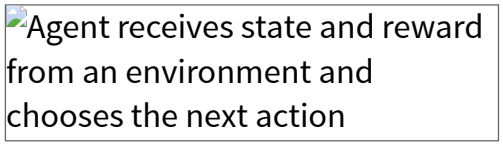
Anomaly detection

Learn typical patterns, then flag unusual cases.
A sudden spending spike might be fraud, or a customer furnishing a house. **Unusual does not mean harmful.**

Without outcome labels, inspecting a neat chart is not enough. Validate the usefulness of the grouping, compression, or alerts.

Reinforcement Learning

Actions Change What Happens Next

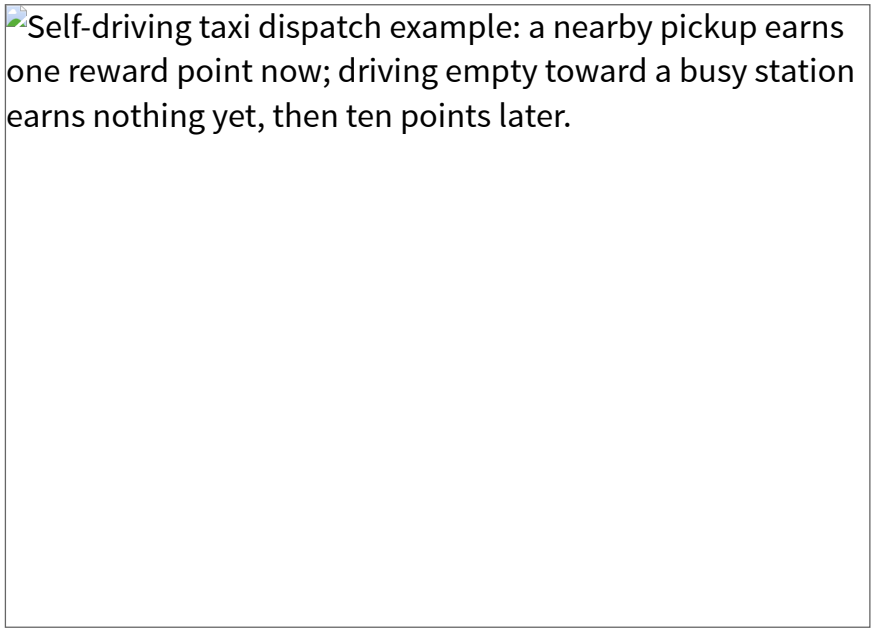


State: where the agent is now.
Action: a move it chooses.
Reward: feedback after acting.
Policy: the rule it uses to choose actions.

In supervised learning, we have example answers. In RL, the agent learns a policy from **experience and rewards**.

Reward Now Versus Reward Later

A self-driving taxi can earn more later by driving empty now.



Take a nearby pickup

Earn **+1 now** in this toy example.

Head to the busy station

Earn **0 while repositioning**, then **+10 on the next step**.

With a discount factor of **0.9**, that later reward counts as **9 today**. It still beats 1.

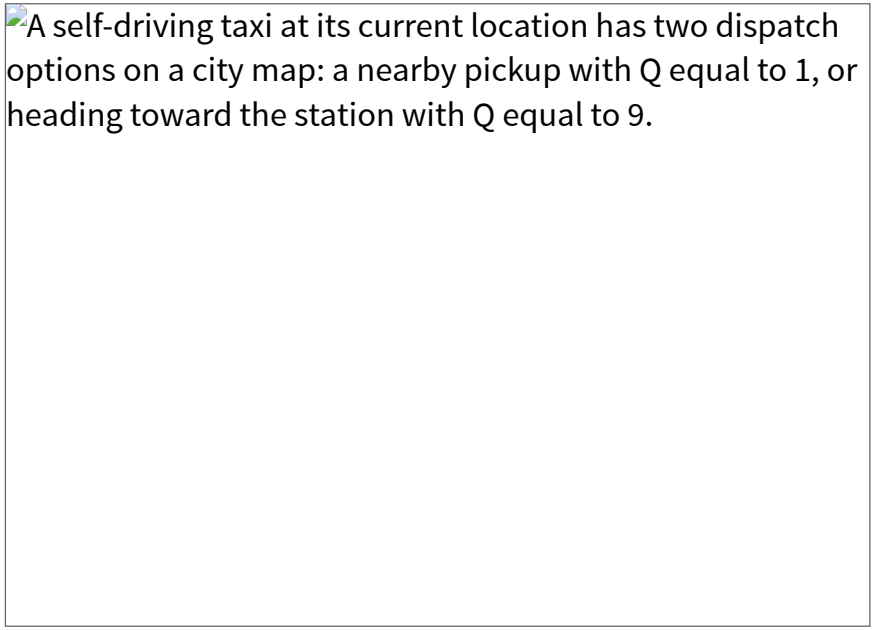
The objective is expected cumulative reward, not the biggest reward on the next move.

Credit assignment: the empty drive helped make the later pickup possible.

Simplified dispatch example with invented reward points.

Q-Learning Stores a Value for Each Action

The taxi learns a forecast for each dispatch choice.



State: the situation now

Where the taxi is, the time, and nearby demand.

Action: what to do next

Take the nearby pickup or head to the station.

Q-value: how worthwhile it looks

$Q(s, a)$ estimates **total discounted reward** from that choice, then following the best-known choices.

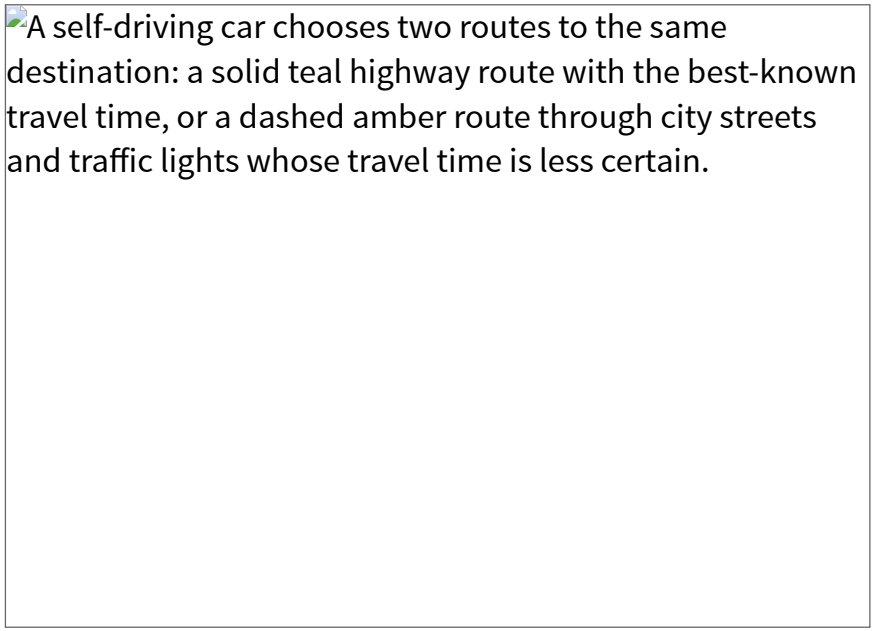
After a move: adjust its value toward **reward received** + discounted best value at the next state. At the end, there is no future value.

Policy: choose the highest value, except when exploring. Here, head to the station. A Q-table needs no neural network.

Same simplified example. These values are learned estimates.

Exploration Collects New Experience

If the car always takes the highway, it may never discover a faster route.



Exploit: use the best estimate

Take the highway. Past trips suggest it is **usually fastest in these conditions**.

Explore: collect experience

Sometimes try the side roads. Traffic and lights may make the trip **faster or slower**.

The actual travel time helps **update the car's route estimates** for future trips.

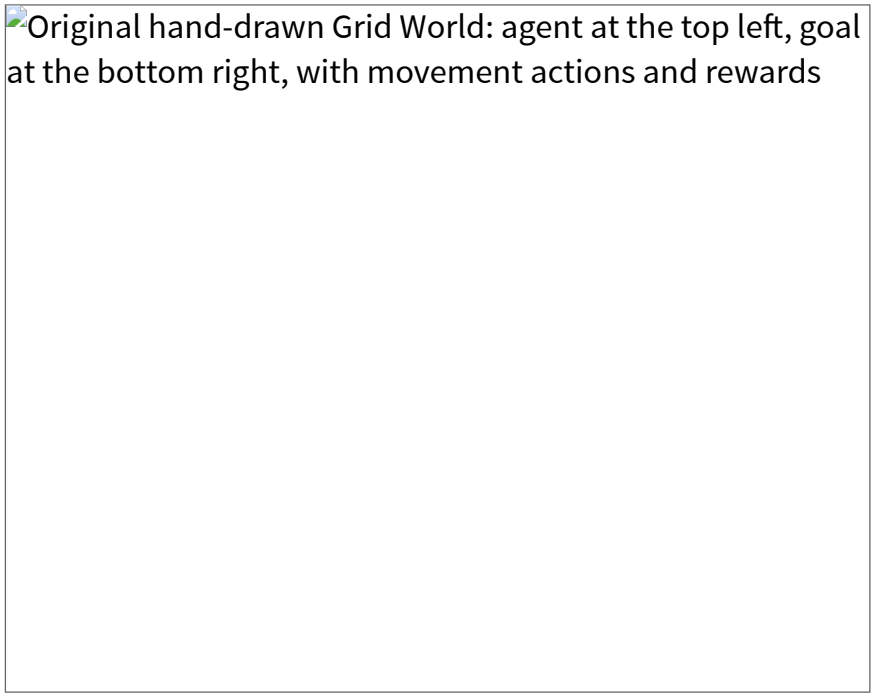
Epsilon-greedy: with probability ϵ, choose a random action; otherwise choose the best known action.

With $\epsilon = 0.2$, about 20% of decisions use a random choice.

Too little exploration can miss better options; too much can reduce current reward.

Simplified navigation example: both routes reach the same destination.

The Grid World Challenge



We program the environment and its rewards. **The agent must learn which actions lead to the goal.**

Try It: Train a Grid World Policy

Start at the top left. Goal: bottom right. **+10 at the goal, −0.1 per other step.**

Exploration 20% | Train 1 episode | Train 50 episodes | Watch policy | Reset

START ? 0.00	(0, 1) ? 0.00	(0, 2) ? 0.00
(1, 0) ? 0.00	(1, 1) ? 0.00	(1, 2) ? 0.00
(2, 0) ? 0.00	(2, 1) ? 0.00	(2, 2) GOAL

Arrows: best known action · number: best Q-value · orange dot: agent

The learned Q-table

Training episodes **0**

Last 50 mean return —

Cell (0, 0)

Up	Right	Down	Left
0.00	0.00	0.00	0.00

Train the zero-initialized table, then watch its greedy policy.

Learning rate α = 0.1 · discount γ = 0.9. Training episodes stop at the goal or a 100-step safety cap.

Train, then watch the policy. Reset and compare exploration at 0% and 20%. Inspect a cell's four Q-values.

The Reward Defines What the Agent Optimizes

What we want	A weak reward	What can go wrong
Finish a maze	+1 for every move	Keep moving without finishing
Resolve support issues	Reward closing tickets	Close tickets before solving them
Drive safely	Reward speed alone	Take dangerous shortcuts

Evaluate the behavior, not just the reward curve. Design a safe environment for exploration.
A software agent using tools is not necessarily trained with RL. RL is a **learning method**, not a synonym for an autonomous workflow.

A Q-Table Cannot List Every Game Screen

Small Grid World	Pac-Man from pixels
9 states × 4 actions = **36 values**	Vast numbers of possible screen configurations
Store each value in a table	Learn a function that estimates values
Look up $Q(s, a)$	Input frames → **neural network** → one Q-value per action

Deep Q-Network (DQN) uses a neural network in place of the Q-table.
The RL objective stays the same. The way we represent action values changes.

End Project: Pac-Man DQN

Train an Agent to Play Ms. Pac-Man



Open the Pac-Man project on GitHub →
Open the notebook in Google Colab →
Run it in **Google Colab**, or clone the project for **Jupyter or VS Code**.

1. Choose **exploration**, **episodes**, and **learning rate**.
2. Select **Run All** to train the ready-made agent.

3. Watch gameplay and compare the before/after scores.

Using **Claude Code or Codex**? The notebook tells it to ask for your three choices before training.

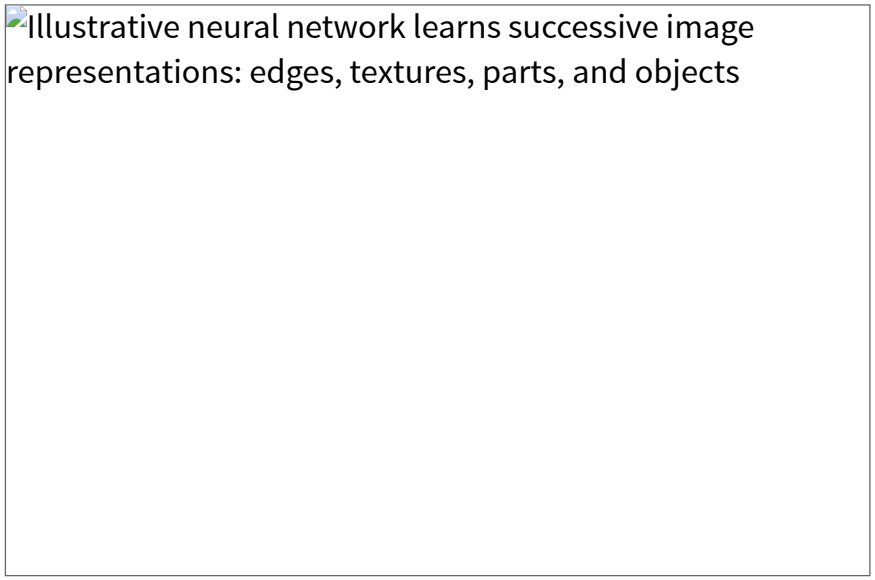
Show What the Agent Actually Learned

Pac-Man project on GitHub → · **Open in Google Colab →** · **Assignment instructions (Google Doc) →**
Your repository should include:

- Your notebook and the **three hyperparameters** you chose.
- An untrained GIF, later checkpoints, and your best gameplay GIF.
- A training plot and a comparison of **five evaluation games** with the untrained baseline, using the same evaluation settings.
- A short explanation of the state, actions, reward, and one observed limitation.

Lower DQN loss does not guarantee better play. Learning speed varies; report what happened, including failed runs.
Submit your GitHub repository →

Class 4: Learn the Features Too



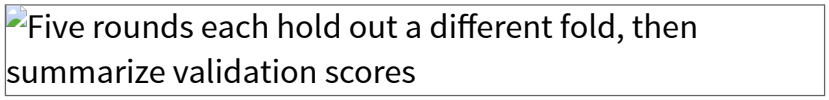
Today: choose features, fit parameters, evaluate on unseen cases, learn actions from rewards.

Next: how layers learn representations, how backpropagation trains them, and how the same ideas lead to embeddings and transformers.

Class 4: Deep Learning, Embeddings & Transformers →

Optional Self-Study

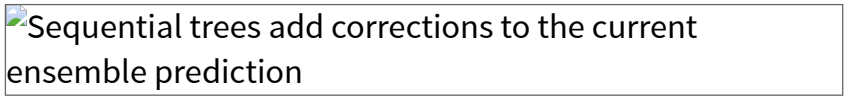
Cross-Validation Rotates the Held-Out Fold SELF-STUDY



Train on four folds; validate on the fifth. Rotate and compare the **average and spread**.
Use these scores to choose settings. Keep a separate **final test set**. Use time-aware or group-aware folds when rows are related.

Boosting Adds Trees That Reduce Remaining Error

SELF-STUDY

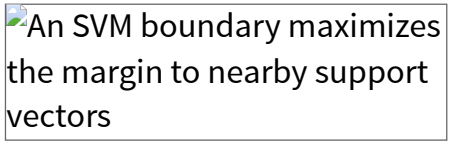


Forest: fit many trees independently, then combine. **Boosting:** add a tree that improves the current prediction, then repeat. The number of trees, their depth, and the learning rate still need validation. Diagram error values are illustrative.

Nearest Neighbors and SVMs SELF-STUDY

Nearest neighbors: predict from similar examples. The choice of distance and feature scaling matters.

Support vector machine: fit a boundary with a wide margin. Kernels can represent nonlinear boundaries.



Both need validation. Neither is automatically the best choice because of its mathematical form.

Calibration Checks Probability Estimates SELF-STUDY

Among cases assigned roughly **70% renewal probability**, about 70% should renew if the model is calibrated for that population.

Good ranking	Good calibration
Higher scores tend to belong to positive cases	Stated probabilities match observed frequencies
Useful for ordering a queue	Useful when probabilities enter a cost calculation

A model can rank well but be overconfident. Check calibration on held-out data and again as conditions change.

Monitor the Model After Launch SELF-STUDY

Monitor	Why
Input distributions and missingness	Tracking or the customer population may have changed
Outcomes once labels arrive	Useful predictions may have become inaccurate
Errors by relevant group	Overall improvement can hide who loses out
Human overrides and review volume	The policy may be overwhelming the team

Assign an owner and a response: investigate, change a threshold, retrain, or roll back. Retraining itself still requires evaluation.

Validation Error in Python SELF-STUDY

Fit increasingly flexible curves. Compare training and validation error on this synthetic dataset.

Python Code | Start Over | Run Code

```python
import numpy as np
rng = np.random.default_rng(0)
x = np.linspace(0, 1, 30)
y = np.sin(2 * np.pi * x) + rng.normal(0, .35, 30)
ids = rng.permutation(30)
train, val = ids[:20], ids[20:]
for degree in [1, 3, 5, 9, 14]:
    fit = np.polynomial.Polynomial.fit(x[train], y[train],
    train_mse = np.mean((fit(x[train]) - y[train]) ** 2)
    val_mse = np.mean((fit(x[val]) - y[val]) ** 2)
    print(f"degree {degree:2}: train={train_mse:.3f}, vali
```

```
_pyodide_editor_1 = ▸Object {code: null, options: Object, indicator: Ge}
```

Which degree would you investigate further? These validation rows guide the choice; they are not an untouched final test set.

Cross-Validate the Renewal Rule SELF-STUDY

Choose a threshold on four accounts, then evaluate on the held-out pair. Rotate the pair.

Python Code | Start Over | Run Code

```
usage = [2, 3, 5, 7, 8, 10]
renewed = [0, 0, 0, 1, 1, 1]
for held_out in [(0, 1), (2, 3), (4, 5)]:
    train = [i for i in range(6) if i not in held_out]
    def accuracy(t, ids):
        return sum((usage[i] >= t) == renewed[i] for i in i
    best_t = max([4, 6, 8], key=lambda t: accuracy(t, trair
    print(f"held out {held_out}: threshold={best_t}, accura
```

_pyodide_editor_2 = ▸Object {code: null, options: Object, indicator: Ge}

Add a low-usage account that renewed and repeat. Six rows demonstrate the procedure; a real evaluation needs representative data and appropriate splits.

The Q-Learning Update SELF-STUDY

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s,a) \right]$$

Suppose current $Q = 2$, reward $r = 0$, best next Q-value $= 5$, $\gamma = 0.9$, and learning rate $\alpha = 0.1$.

Target = 0 + 0.9 × 5 = 4.5. New Q = 2 + 0.1 × (4.5 − 2) = **2.25**.

For a terminal transition, target = reward only. There is no next-state value to add.

Why DQN Uses Replay and a Target Network SELF-STUDY

Component	Purpose
Frame stack	Gives the network information about motion
Experience replay	Samples past transitions to reduce correlation and reuse experience
Target network	Provides a slower-changing target for value updates
Exploration schedule	Balances trying new actions with using current estimates

The loss measures error against a **value target**, not whether the game was played well. Track evaluation reward and watch behavior too.

Pac-Man Notebook Setup SELF-STUDY

Download the notebook → · **Setup and publishing guide**

- **Colab:** File → Upload notebook. Select a GPU under Runtime → Change runtime type if available.
- **Local:** open the notebook in Jupyter or VS Code and select a Python kernel.
- Choose your **exploration**, **episodes**, and **learning rate**, then **Run All**.
- Start with **five episodes** to check setup. Download the results ZIP when finished.

  The notebook handles setup, training, GIFs, plots, checkpoints, and evaluation. Read the code to connect each step to the RL concepts.
  Optional: Build an agent from scratch with AI.

Speaker notes